A

Project Report

on

EchoSafe – Real-Time Distress Sound Detection and Alert System

submitted as partial fulfillment for the award of

BACHELOR OF TECHNOLOGY

DEGREE

SESSION 2025-26

in

Computer Science and Engineering (Artificial Intelligence & Machine Learning)

by

Prakhar Dhimaan (2200291530080)

Siddharth Singh (2200291530112)

Neelabh Srivastava (2200291530071)

Sahil Kumar Jha (2200291530089)

Under the supervision of

Ms. Nidhi Singh (Assistant Professor)

Krishna Institute of Engineering & Technology (KIET), Ghaziabad, Delhi-NCR, Uttar Pradesh, India

March, 2026

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Prakhar Dhimaan (2200291530080) with Signature

Siddharth Singh (2200291530112) with Signature

Neelabh Srivastava (2200291530) with Signature

Sahil Kumar Jha (2200291530) with Signature

Date:

CERTIFICATE

This is to certify that Project Report entitled "Robust Real-Time Scream Detection Using Hybrid CNN-Transformer and MLP-SVM Models" which is submitted by Prakhar Dhimaan, Siddharth Singh, Neelabh Srivastava, and Sahil Kumar Jha in partial fulfillment of the requirement for the award of degree B. Tech. in Department of CSE (AIML) of Dr. A.P.J. Abdul Kalam Technical University, Lucknow is a record of the candidates own work carried out by them under my supervision. The matter embodied in this report is original and has not been submitted for the award of any other degree.

Ms. Nidhi Singh **Dr. Rekha Kashyap**

(Designation) **Dean – CSE (AI/AI&ML)**

Date:

ACKNOWLEDGEMENT

It gives us a great sense of pleasure to present the report of the B.Tech Project undertaken during the Final Year. We owe a special debt of gratitude to Ms. Nidhi Singh, Department of Computer Science & Engineering (Artificial Intelligence & Machine Learning), KIET Group of Institutions, Ghaziabad, for her constant support and guidance throughout the course of our work. Her sincerity, thoroughness, and perseverance have been a constant source of inspiration for us. It is only through her cognizant efforts that our endeavors have seen the light of day.

We also take the opportunity to acknowledge the contribution of Dr. Rekha Kashyap, Dean of the Department of Computer Science & Engineering (AI/AI&ML), KIET, Ghaziabad, for his full support and assistance during the development of the project. We also do not like to miss the opportunity to acknowledge the contribution of all the faculty members of the department for their kind assistance and cooperation during the development of our project.

Date:
Prakhar Dhimaan (2200291530080) with Signature

Siddharth Singh (2200291530112) with Signature

Neelabh Srivastava (2200291530) with Signature

Sahil Kumar Jha (2200291530) with Signature

ABSTRACT

The increasing demand for public safety and real-time surveillance systems has led to the development of intelligent audio-based monitoring solutions. Among various distress signals, human screams are one of the most critical indicators of emergencies such as violence, accidents, or medical crises.

This project presents a **robust real-time scream detection system** based on a hybrid deep learning architecture that integrates **CNN-Transformer and MLP-SVM models**. The proposed system processes audio signals using two parallel feature extraction pathways. The first pathway utilizes Mel-spectrograms processed through convolutional neural networks and transformer encoders to capture high-level spectral and temporal patterns. The second pathway extracts MFCC-based features and processes them through a multilayer perceptron model.

A fusion mechanism combines the outputs of both pathways to improve classification performance. The system is trained on a large dataset consisting of scream and non-scream audio samples under various noise conditions.

Experimental results demonstrate that the hybrid model achieves high accuracy (~97–98%) and performs effectively even in noisy environments. The system is capable of generating real-time alerts, making it suitable for applications in surveillance systems, smart cities, and emergency response frameworks.

Page No.

DECLARATION……. ii
CERTIFICATE…… iii
ACKNOWLEDGEMENT..……. iv
ABSTRACT……..... v
LIST OF FIGURES……. ix
LIST OF TABLES…… xi
LIST OF ABBREVIATIONS……. xii

CHAPTER 1 (INTRODUCTION)……. 1
1.1 Background of the Study…… 1
1.2 Problem Statement……. 6
1.3 Objectives……... 7
1.4 Project Scope…….. 8
1.5 Organization of the Report……. 9

CHAPTER 2 (LITERATURE RIVIEW)……. 10
2.1. Overview of Audio Surveillance and Scream Detection……... 10
2.2. Evolution of Deep Learning in Audio Classification…… 12
2.3. Review of Existing AI-Based Scream Detection Platforms…….. 14
2.4. Feature Extraction Techniques in Audio Processing……. 16
2.5. Research Gap Identification…….. 18

CHAPTER 3 (PROPOSED METHODOLOGY) …..... 20

3.1. Overall System Architecture……..

3.2. Audio Preprocessing and Feature Extraction Framework……. 20
3.3. CNN-Transformer Branch……. 22
3.4. MLP-SVM Branch……. 24
3.5. Hybrid Decision Fusion Mechanism……. 26
3.6. Dataset Description and Preparation…….. 27
3.7. Real-Time Deployment Pipeline……... 29
30

CHAPTER 4 (RESULTS AND DISCUSSION) 31
4.1. Development Technology Stack……... 31
4.2. Detailed Implementation Process…….
4.3. System Testing and Validation……. 33
4.4. Performance Evaluation Metrics…….. 35
4.5. Model Comparison and Analysis……. 37
4.6. Scenario-Based Evaluation…….. 38
4.7. Discussion of Results……...
40

CHAPTER 5 (CONCLUSIONS AND FUTURE SCOPE)..... 41

7.1. Conclusion..... 41
7.2. Future Scope..... 42

REFERENCES……. 43
APPENDEX1……... 44

RESEARCH PAPER/PATENT
PROOF OF SUBMISSION/ACCEPTANCE/PUBLICATION
PLAGIARISM REPORT

LIST OF FIGURES

Figure No.	Description	Page No.
3.1	Overall System Architecture Diagram	20
3.2	Audio Preprocessing and Feature Extraction Pipeline	22
3.3	CNN-Transformer Branch Architecture	25
3.4	MLP-SVM Branch Architecture	26
3.5	Hybrid Intermediate Fusion Network	28

LIST OF TABLES

Table. No.	Description	Page No.
2.1	Comparison of Existing Scream Detection Systems	16
4.1	Development Technology Stack	31
4.2	Functional Testing Scenarios and Results	36
4.3	Performance and Latency Metrics	34
4.4	Model Performance Comparison	38
4.5	Scenario-Based Evaluation Results	39
4.6	Confusion Matrix and Performance Metrics	40

LIST OF ABBREVIATIONS

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
ML	Machine Learning
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
CRNN	Convolutional Recurrent Neural Network
MLP	Multilayer Perceptron
SVM	Support Vector Machine
MFCC	Mel-Frequency Cepstral Coefficients
STFT	Short-Time Fourier Transform
GPU	Graphics Processing Unit
ODE50	Onset Detection Efficiency (within 0.5 seconds)
SNR	Signal-to-Noise Ratio
ReLU	Rectified Linear Unit
AUC	Area Under Curve

CHAPTER 1
INTRODUCTION

1.1 Background of the Study

1.1.1 The Growing Need for Intelligent Audio Surveillance

Public safety has always been a paramount concern for governments, institutions, and communities worldwide. Traditional surveillance systems have relied predominantly on visual technologies — closed-circuit television cameras, motion sensors, and human security personnel — to monitor and respond to threatening situations. While these systems have proven effective in many contexts, they carry inherent limitations in coverage, response speed, and the ability to detect events that occur outside the camera's field of view or in low-visibility conditions.

The 21st century has witnessed an accelerating convergence of artificial intelligence, machine learning, and edge computing technologies, fundamentally transforming the landscape of safety and surveillance systems. Among the most promising developments in this domain is the application of intelligent audio analysis — the ability of computational systems to automatically analyze sound environments and detect specific acoustic events of safety relevance.

Human vocalizations carry rich information about emotional and physiological states. Among all human sounds, screams and distress vocalizations are among the most universally recognized signals of danger, pain, fear, or emergency. Across cultures, languages, and demographics, a scream in a public space or private environment typically indicates that something has gone seriously wrong — a violent attack, a medical emergency, an accident, or a moment of extreme psychological distress. The timely detection of such sounds can enable rapid emergency response, potentially preventing harm or saving lives.

Despite this intuitive importance, automated systems capable of reliably detecting human screams in real-world acoustic environments remain technically challenging to develop and deploy. Screams exhibit enormous acoustic variability across individuals — differing in pitch, duration, intensity, spectral distribution, and temporal dynamics. Real-world

environments further complicate detection through the presence of background noise, overlapping sound events, reverberation, and recording artifacts. Building a system that achieves both high sensitivity and high specificity — detecting true screams while avoiding false alarms from other human vocalizations, alarms, or environmental sounds — requires sophisticated machine learning approaches.

1.1.2 Audio-Based Safety Systems in Modern Infrastructure

Modern urban infrastructure is increasingly integrating intelligent sensing systems beyond traditional visual surveillance. Smart city initiatives around the world are deploying sensor networks capable of monitoring environmental conditions, traffic patterns, energy consumption, and public behavior. Within this framework, acoustic monitoring represents an underutilized but high-potential safety layer.

Audio-based surveillance systems offer several distinct advantages over purely visual approaches. Sound propagates omnidirectionally, meaning a microphone can detect events occurring behind walls, in blind spots, or in areas with no line of sight. Acoustic systems can also operate effectively in low-light or zero-light conditions where cameras fail. Furthermore, audio analysis can detect physiological and behavioral signals — such as screams, gunshots, breaking glass, or aggressive speech — that complement visual detection of physical movement.

Despite these advantages, deployment of intelligent audio surveillance has historically lagged behind visual surveillance, primarily due to the technical challenge of accurate sound classification in noisy real-world environments and concerns about privacy. Recent advances in deep learning, particularly transformer-based architectures and large-scale audio datasets, have dramatically improved the capability of audio classification systems, bringing reliable real-time scream detection closer to practical deployment.

1.1.3 Psychological and Societal Dimensions of Distress Detection

The development of automated distress detection systems carries significant societal implications. Timely detection of human screams can enable faster emergency response in several critical scenarios. In cases of violent assault or robbery, seconds can make a critical difference between intervention and serious harm. In medical emergencies — falls, cardiac

events, or strokes — elderly or vulnerable individuals who are unable to manually call for help may produce distress vocalizations that an automated system could detect and relay to caregivers or emergency services. In campus environments, automated detection of distress signals can complement existing panic button systems, providing broader and more continuous coverage.

From a psychological perspective, the knowledge that intelligent safety systems are monitoring an environment can also produce a deterrence effect, potentially reducing the incidence of crimes in monitored public spaces. For vulnerable populations — children, the elderly, individuals with disabilities — the deployment of reliable scream detection systems can provide an additional layer of assurance to caregivers and families.

However, the ethical dimensions of acoustic surveillance must be carefully considered. Unlike visual surveillance, audio monitoring raises additional privacy concerns due to the ability to capture private conversations. Responsible deployment of scream detection systems requires careful engineering decisions — processing audio locally on edge devices without transmission or storage, detecting only specific acoustic events rather than recording continuous audio, and implementing robust false alarm management mechanisms to avoid unnecessary intrusion or stigmatization.

1.1.4 Acoustic Characteristics of Human Screams

Understanding the acoustic properties of screams is essential for designing effective detection systems. Screams are characterized by several distinctive acoustic features that differentiate them from normal speech or environmental sounds. Compared to conversational speech, screams typically exhibit significantly higher fundamental frequency (pitch), greater vocal intensity, broader harmonic bandwidth, increased spectral flatness, and more abrupt temporal onset patterns.

Research in vocal acoustics has established that scream vocalizations engage different physiological mechanisms than normal speech. The supraglottal vocal tract is typically less constricted during screaming, producing a more open-throated resonance pattern. The subglottal air pressure driving the vocal folds is substantially higher, resulting in greater sound intensity. These physiological differences produce spectral signatures that, while

variable across individuals and contexts, share statistical regularities that machine learning models can learn to identify.

A key challenge in scream detection, however, is that some non-scream vocalizations — particularly loud laughter, cheering, certain musical performances, and cries of excitement — share acoustic characteristics with distress screams. Differentiating between vocalizations of fear and vocalizations of joy remains an open challenge in the field, and one that the proposed hybrid system addresses by leveraging both deep learned representations and compact handcrafted features.

1.1.5 Role of Machine Learning in Audio Event Detection

Machine learning has transformed audio signal processing over the past decade. Traditional approaches to sound event detection relied on handcrafted features — Mel-Frequency Cepstral Coefficients (MFCCs), zero crossing rates, spectral centroids, and Gaussian Mixture Models or Support Vector Machines for classification. While effective in controlled conditions, these methods proved brittle in the presence of significant acoustic variation and background noise.

The advent of deep learning brought convolutional neural networks capable of learning discriminative spectral-temporal patterns directly from raw spectrograms. More recently, transformer architectures — originally developed for natural language processing — have been successfully applied to audio analysis, enabling models to capture long-range temporal dependencies that CNNs alone struggle to model effectively. The combination of CNN-based local feature extraction with transformer-based global context modeling has emerged as one of the most effective architectural paradigms for audio classification tasks.

The proposed system in this project advances this paradigm by combining a CNN-Transformer branch with an MLP branch operating on handcrafted features, creating a hybrid model that benefits from both learned deep representations and carefully engineered acoustic descriptors.

1.2 Problem Statement

The field of automated scream detection faces several interconnected technical and practical challenges that current systems have not fully resolved.

Acoustic Variability: Screams vary dramatically across individuals in terms of pitch, duration, intensity, and spectral composition. A system trained predominantly on one demographic or vocal style may fail to generalize to others, including child screams, culturally specific vocalizations, or screams from individuals with unusual vocal characteristics.

Background Noise Interference: Real-world deployment environments — urban streets, public transit, crowded venues — contain high levels of background noise at varying signal-to-noise ratios. Conventional classifiers trained on clean audio data suffer significant performance degradation in noisy conditions, limiting their practical utility.

Limited Integration of Spectral and Temporal Information: Many existing systems rely either on local spectral patterns captured by CNNs or on long-range temporal modeling through recurrent architectures, but rarely integrate both effectively. Screams involve both characteristic local spectral signatures and temporal dynamics — onset, duration, and decay — that benefit from joint modeling.

False Alarm Management: High false alarm rates pose a critical practical problem for deployed safety systems. Unnecessary alerts desensitize operators, increase response costs, and reduce trust in the system. Achieving high precision alongside high recall in diverse acoustic conditions requires more sophisticated classification approaches than existing single-branch systems typically provide.

Real-Time Processing Constraints: A deployable safety system must process audio streams with low latency — ideally detecting screams within 0.5 seconds of onset — while running on hardware available in real-world surveillance infrastructure, including embedded edge devices with limited computational resources.

Insufficient Dataset Diversity: Many existing scream detection models have been trained on relatively small or homogeneous datasets, limiting their generalization to novel environments and acoustic conditions not represented in training data.

There is therefore a clear need for an intelligent, scalable, robust, and deployable system capable of bridging these technical gaps through an innovative hybrid machine learning architecture.

1.3 Objectives

The primary aim of this project is to design and implement a robust, real-time scream detection system that leverages a hybrid deep learning architecture integrating CNN, Transformer, MLP, and SVM components. The specific objectives are as follows:

1. To Design a Hybrid CNN-Transformer + MLP Architecture The first objective is to develop a novel dual-branch neural network architecture in which a CNN-Transformer branch learns high-level spectrotemporal representations from Mel-spectrograms, while an MLP branch processes compact handcrafted MFCC-based features. The integration of these complementary representations aims to produce a system more robust than any single-branch approach.

2. To Implement Effective Audio Preprocessing and Feature Extraction The project aims to develop a complete audio preprocessing pipeline capable of transforming raw audio waveforms into both Mel-spectrogram representations and MFCC-based feature vectors suitable for input to the respective model branches. This pipeline must operate in real-time with minimal latency.

3. To Develop an Intermediate Fusion Mechanism A core objective is to implement and evaluate an intermediate fusion network that concatenates pooled feature vectors from both branches and learns a joint representation for final classification. The fusion network must improve upon individual branch performance without introducing excessive computational overhead.

4. To Evaluate and Compare an SVM-Based Classification Alternative The project includes exploration of a Support Vector Machine classifier using MLP branch features as an alternative to end-to-end learned classification. Comparative evaluation against the MLP branch provides insight into the relative merits of traditional and deep learning classifiers on MFCC-based features.

5. To Train and Evaluate the System on a Large-Scale Diverse Dataset The system must be trained and evaluated on a comprehensive dataset comprising over 11,900 labelled scream samples and an equivalent number of non-scream samples, including diverse noise conditions at multiple signal-to-noise ratios, to ensure robust generalization.

6. To Achieve Real-Time Detection Performance The system must meet real-time processing requirements, achieving scream onset detection within 0.5 seconds in at least 90% of cases, as measured by the ODE50 metric, while maintaining end-to-end inference times compatible with practical streaming deployment.

7. To Evaluate Scenario-Based Robustness The project aims to conduct systematic evaluation across distinct real-world deployment scenarios including quiet indoor environments, noisy street conditions, crowded public event settings, and false-alarm conditions, providing a comprehensive picture of practical system performance.

8. To Analyze Component Contributions Through Ablation Studies A dedicated component analysis objective involves replacing the Transformer encoder with a BiLSTM network to isolate the contribution of the attention mechanism to system performance, enabling principled architectural decisions.

9. To Discuss Responsible Deployment Considerations The project aims to address the ethical, privacy, and operational considerations associated with deploying automated acoustic surveillance systems, providing a responsible engineering framework for future deployment decisions.

10. To Document the Complete Development and Evaluation Lifecycle The final objective is to produce a comprehensive technical report documenting all aspects of the system development, from literature review and methodology through implementation, results, and recommendations for future work.

1.4 Project Scope

The scope of this project encompasses the following:

- Design and implementation of a dual-branch hybrid neural network architecture for audio classification

- Development of a complete audio preprocessing pipeline including Mel-spectrogram generation and MFCC feature extraction
- Implementation of an intermediate fusion network for combining branch outputs
- Training and evaluation on a large-scale scream detection dataset with diverse noise augmentation
- Systematic comparison against five baseline models including MLP, SVM, CNN-Transformer, ResNet, and CRNN
- Scenario-based evaluation across quiet, noisy, and crowded acoustic environments
- Ablation study comparing Transformer and BiLSTM temporal modeling
- Discussion of deployment considerations for urban surveillance, campus security, home safety, and emergency response contexts

---

The project does not aim to build a complete end-to-end surveillance product but rather demonstrates the core machine learning architecture and its performance characteristics, serving as a validated foundation for integration into larger safety systems.

1.5 Organization of the Report

This report is organized as follows. Chapter 2 presents a comprehensive literature review covering the evolution of audio-based safety systems, deep learning for audio classification, existing scream detection platforms, feature extraction techniques, and identified research gaps. Chapter 3 describes the proposed system architecture and methodology in detail, including the CNN-Transformer branch, MLP-SVM branch, fusion mechanism, and dataset preparation. Chapter 4 covers the implementation environment, detailed implementation process, testing procedures, and comprehensive results. Chapter 5 presents conclusions, acknowledges limitations, and proposes directions for future enhancement.

CHAPTER 2

LITERATURE REVIEW

2.1 Overview of Audio Surveillance and Scream Detection

Audio-based surveillance and sound event detection have evolved from simple threshold-based acoustic alarms to sophisticated machine learning systems capable of classifying

complex real-world sound environments. The history of this evolution reflects broader trends in signal processing, pattern recognition, and deep learning research.

Early acoustic surveillance systems operated on simple energy-based detection principles. A sound event was flagged when the root mean square energy of a microphone signal exceeded a predetermined threshold. While computationally trivial, such systems produced unacceptably high false alarm rates in anything but quiet controlled environments, rendering them impractical for real-world deployment.

The introduction of handcrafted feature-based approaches in the 1990s and early 2000s marked a significant advance. Researchers began applying statistical pattern recognition techniques — particularly Gaussian Mixture Models and Hidden Markov Models — to features derived from the frequency-domain representation of audio signals. Mel-Frequency Cepstral Coefficients, derived from the perceptually motivated Mel-frequency scale, became the dominant feature representation for speech and audio classification tasks, a role they have maintained across decades of methodological evolution.

The application of these techniques to scream detection specifically began gaining attention in the context of public safety and emergency response research in the mid-2000s. Studies demonstrated that screams possessed distinctive MFCC signatures that could be reliably separated from normal speech and common environmental sounds under controlled conditions. However, performance degraded substantially when systems were evaluated in realistic noisy environments representative of actual deployment conditions.

2.1.1 The Significance of Scream Detection in Safety Systems

Among the many categories of audio events relevant to public safety, human screams occupy a particularly significant position. Unlike gunshots, glass breaking, or alarms — which are predominantly mechanical sounds — screams are human vocalizations that directly indicate human distress. Their detection therefore carries immediate implications for human welfare.

From a signal processing perspective, scream detection is challenging for several reasons beyond the baseline problem of audio classification in noise. The acoustic space of human vocalizations is exceptionally broad. Normal human vocal production encompasses speech, laughter, crying, singing, shouting, and screaming, with substantial acoustic overlap between

these categories. A scream of fear and a scream of excitement, for instance, may be acoustically indistinguishable without contextual information.

Furthermore, the population of potential scream producers is highly diverse. Screams from children, women, and men differ substantially in fundamental frequency range and harmonic structure. Screams produced in different emotional states — terror, pain, anger, surprise — exhibit different spectral profiles. A robust scream detection system must generalize across this entire space of acoustic variability.

2.1.2 Key Acoustic Features of Distress Screams

Research in vocal acoustics and affective computing has identified several acoustic features that consistently differentiate distress screams from other human vocalizations and environmental sounds. These features form the basis for both handcrafted feature engineering and serve as a reference for evaluating what deep learning models implicitly learn.

The roughness or harshness of a scream vocalization, arising from irregular vocal fold vibration patterns, produces distinctive acoustic signatures in the modulation frequency range of approximately 30-150 Hz. This roughness quality, measured through various acoustic metrics, has been identified as a cross-cultural marker of danger signaling in animal and human vocalizations alike.

High spectral flatness — the ratio of geometric mean to arithmetic mean of the power spectrum — indicates a noise-like rather than tonal sound, and screams typically exhibit higher spectral flatness than speech. High overall intensity, broad harmonic bandwidth, and abrupt temporal onset are additional characteristic features that machine learning systems can exploit for classification.

2.2 Evolution of Deep Learning in Audio Classification

2.2.1 From Handcrafted Features to Learned Representations

The transition from handcrafted feature engineering to end-to-end deep learning has been one of the defining methodological shifts in audio processing research over the past decade. In the handcrafted paradigm, domain experts designed features believed to capture relevant acoustic properties, followed by conventional classifiers such as SVMs or GMMs. While effective

when the relevant features were well understood, this paradigm was fundamentally limited by the ability of human designers to anticipate the full complexity of real-world audio variation.

Deep learning offered a radical alternative: learn feature representations directly from data, allowing the model itself to discover which aspects of the audio signal are most discriminative for a given task. Convolutional Neural Networks, originally developed for image recognition, proved highly effective when applied to two-dimensional time-frequency representations of audio — particularly spectrograms and Mel-spectrograms — which could be treated as images with rich local structure.

Early CNN-based audio classification systems demonstrated that convolutional filters applied to Mel-spectrograms could automatically learn to detect local spectral-temporal patterns corresponding to acoustic events of interest, without requiring explicit hand-engineering of features. These early successes spurred rapid growth in deep learning applications for audio, including music classification, speech recognition, environmental sound detection, and eventually audio surveillance.

2.2.2 Recurrent Architectures for Temporal Modeling

While CNNs proved effective at capturing local spectral patterns, audio signals are inherently sequential, with important information encoded in long-range temporal dependencies. The onset and decay dynamics of a scream, for instance, unfold over time in ways that local convolutional filters cannot fully capture. This motivated the application of recurrent neural networks — particularly Long Short-Term Memory (LSTM) networks — to audio classification.

LSTM-based models process audio representations as sequences of feature vectors, maintaining hidden state across time steps to capture temporal context. CRNN (Convolutional Recurrent Neural Network) architectures that combine CNN feature extraction with LSTM temporal modeling became a widely adopted paradigm for audio event detection, demonstrating strong performance on benchmark datasets across multiple audio classification tasks.

However, LSTMs have well-documented limitations. Their sequential processing structure makes parallelization difficult, increasing training time. They can struggle with very long

sequences, and their ability to model dependencies across many time steps can degrade despite the gating mechanisms designed to address the vanishing gradient problem.

2.2.3 Transformer Architectures for Audio

The transformer architecture, introduced by Vaswani et al. for neural machine translation and subsequently applied across natural language processing tasks, offers a fundamentally different approach to sequence modeling. Rather than processing sequences step-by-step as LSTMs do, transformers apply multi-head self-attention mechanisms that directly model relationships between all positions in a sequence simultaneously, enabling efficient parallelization and effective capture of long-range dependencies.

The application of transformer architectures to audio began gaining momentum with the introduction of models such as Audio Spectrogram Transformer (AST), which demonstrated that pure attention-based models could match or exceed CNN-based approaches on audio classification benchmarks when applied to spectrogram inputs. Subsequently, hybrid CNN-Transformer architectures emerged that used CNNs to extract local spectral features and transformers to model temporal relationships between these features across the audio window, combining the complementary strengths of both architectural families.

For scream detection specifically, Kim et al. demonstrated that a CNN-Transformer framework trained on a large-scale scream dataset achieved approximately 97.6% accuracy and an F1-score of 0.982, establishing a strong baseline that the proposed hybrid system in this project aims to match and extend.

2.2.4 Transfer Learning in Audio Classification

Transfer learning — the practice of initializing a model with weights pre-trained on a large related dataset before fine-tuning on a target task — has become standard practice in deep learning across domains. In audio, models pre-trained on large general sound event datasets have been shown to transfer effectively to specialized tasks including scream detection, accelerating convergence and improving generalization especially when target task training data is limited.

The proposed system in this project follows this established practice, initializing the CNN-Transformer branch with weights pre-trained on a general audio event classification task before fine-tuning on the scream detection dataset.

2.3 Review of Existing AI-Based Scream Detection Platforms

2.3.1 MFCC-Based Classical Systems

The earliest approaches to automated scream detection employed MFCC features combined with classical classifiers. Chung and Chung compared SVM and GMM classifiers on MFCC features for scream detection, finding SVMs superior but noting substantial performance degradation in the presence of background noise. These systems established MFCC-based representations as the dominant feature type for this task and provided important benchmarks, but their noise sensitivity limited their practical applicability.

Guo et al. proposed a scream detection system based on sound event partitioning with robust MFCC features, achieving performance improvements through careful feature engineering and preprocessing. However, the system's reliance on handcrafted features continued to limit generalization to novel environments.

2.3.2 CNN-Based Deep Learning Systems

The introduction of CNN-based approaches to scream detection represented a significant methodological advance. Laffitte et al. conducted a systematic comparison of MLP, CNN, RNN, and CRNN architectures for scream and shout detection in public transportation audio, finding deep architectures consistently superior to handcrafted feature approaches. Their work demonstrated detection rates of 82.2% in realistic noisy public transport audio, highlighting both the promise and the remaining challenges of deep learning approaches in practical deployment conditions.

Lim et al. developed a real-time scream detection system for smart surveillance applications using CNN architectures trained with noise augmentation, demonstrating that training on noise-corrupted data significantly improved robustness in urban acoustic conditions. Gumpp et al. extended this line of work, exploring various CNN configurations for scream detection in noisy environments and establishing that deeper convolutional architectures with appropriate regularization consistently outperformed shallower models.

2.3.3 CNN-Transformer Hybrid Systems

Kim et al. introduced what is arguably the most significant recent advance in scream detection research, presenting both a large-scale scream dataset of 11,921 labelled samples and a CNN-Transformer framework trained upon it. Their system achieved approximately 97.6% accuracy and an F1-score of 0.982, establishing new state-of-the-art performance. The combination of CNN local feature extraction with Transformer global temporal modeling proved highly effective, and the large dataset scale enabled robust generalization.

The proposed system in this project builds directly upon this foundation, extending the CNN-Transformer approach with an additional MLP branch processing handcrafted features and an intermediate fusion mechanism, aiming to further improve performance particularly in borderline and high-noise scenarios where the CNN-Transformer alone may be uncertain.

2.3.4 Wave-U-Net and Noise-Robust Architectures

Hayasaka et al. explored a Wave-U-Net architecture for scream detection, approaching the problem through the lens of audio source separation rather than direct classification. By applying a noise-robust source separation front-end before classification, their system demonstrated strong performance on a smaller dataset of screaming and distress sounds, particularly in high-noise conditions. This work highlighted the potential value of explicitly modeling and compensating for noise as a preprocessing step, rather than relying solely on augmentation during training.

2.3.5 Comparison Table of Existing Systems

SYSTEM	FEATURES	CLASSIFIER	DATASET SIZE	ACCURACY	NOISE ROBUSTNESS
CHUNG & CHUNG (2017)	MFCC	SVM/GMM	Small	~85%	Low
LAFFITTE ET AL. (2019)	Spectrogram	MLP/CNN/RNN/CRNN	Medium	82.2%	Medium
GUMPPET AL. (2020)	Spectrogram	CNN	Medium	~91%	Medium-High
LIM ET AL. (2019)	Spectrogram	CNN	Medium	~93%	Medium-High
KIM ET AL. (2024)	Mel-Spectrogram	CNN-Transformer	11,921	97.6%	High
HAYASAKA ET AL. (2024)	Waveform	Wave-U-Net	Small	N/A	High
PROPOSED SYSTEM	Mel-Spectrogram + MFCC	Hybrid CNN-Transformer + MLP	~24,000	97.8%	Very High

Table 2.1: Comparison of Existing Scream Detection Systems

2.4 Feature Extraction Techniques in Audio Processing

2.4.1 Mel-Frequency Cepstral Coefficients (MFCCs)

MFCCs remain among the most widely used features in audio and speech processing despite decades of methodological evolution. The computation of MFCCs involves a series of transformations designed to mimic the perceptual processing of the human auditory system. Audio frames are first transformed to the frequency domain via Short-Time Fourier

Transform (STFT). The resulting power spectrum is mapped to the Mel frequency scale — a perceptually motivated scale that reflects the non-linear frequency resolution of the human cochlea — through application of a triangular filter bank. The log of the filter bank energies is then transformed via Discrete Cosine Transform (DCT) to produce the final cepstral coefficients.

The resulting MFCCs provide a compact representation of the spectral envelope of an audio frame, capturing the most perceptually relevant aspects of the sound's frequency content in a relatively low-dimensional feature vector. For scream detection, typically 13-40 MFCC coefficients are extracted per frame, often supplemented with delta (first derivative) and delta-delta (second derivative) coefficients capturing the temporal dynamics of spectral change.

2.4.2 Mel-Spectrograms

While MFCCs provide a compact summarized representation, Mel-spectrograms preserve the full time-frequency content of the audio signal on the Mel scale, providing richer input for deep learning models capable of learning from high-dimensional representations. A Mel-spectrogram is computed by applying the same Mel-scale filter bank as in MFCC computation but without the subsequent DCT, preserving the spatial structure of the time-frequency plane.

Mel-spectrograms are well-suited as input to convolutional networks because their two-dimensional structure — frequency on one axis, time on the other — closely resembles images, allowing convolutional feature detectors to learn local spectral-temporal patterns analogously to how image CNNs learn local visual patterns. The choice of window size, hop size, and number of Mel filters involves trade-offs between temporal resolution, frequency resolution, and computational efficiency.

2.4.3 Supplementary Spectral Features

Beyond MFCCs and Mel-spectrograms, several additional spectral features provide complementary information useful for audio classification. Spectral roll-off captures the frequency below which a specified percentage of the total spectral energy is contained, providing information about spectral shape. Spectral flux measures the rate of change of the

power spectrum between successive frames, capturing the temporal dynamics of spectral evolution. Spectral entropy quantifies the disorder or unpredictability of the power spectrum, distinguishing tonal from noise-like sounds. Zero crossing rate measures the rate at which the signal changes sign, providing a simple indicator of noisiness.

In the proposed system, these supplementary features are aggregated alongside MFCCs over one-second windows using their mean and variance, producing a compact feature vector that complements the high-dimensional Mel-spectrogram input to the CNN-Transformer branch.

2.4.4 Noise Augmentation Strategies

A critical challenge in training robust audio classification systems is preparing models to generalize to noise conditions not present in clean training data. Several standard augmentation strategies have proven effective. Additive noise augmentation — mixing training samples with recorded background sounds at various signal-to-noise ratios — is the most widely used approach. Time shifting — randomly shifting the temporal position of audio events within the analysis window — teaches models to detect sounds regardless of their position in the input. Pitch shifting — applying small random changes to fundamental frequency — increases robustness to vocal variability across speakers.

The proposed system employs additive Gaussian noise and background audio augmentation during training, as well as small time-shifts, preparing the model to handle diverse real-world noise conditions.

2.5 Research Gap Identification

Despite significant progress in scream detection research, several important research gaps remain that motivate the proposed work.

1. Limited Integration of Deep and Handcrafted Features Most existing systems rely either exclusively on deep learned representations (CNN-Transformer, CRNN) or exclusively on handcrafted features (MFCC-SVM). The potential synergy between these complementary approaches — combining the discriminative power of learned representations with the noise robustness of carefully engineered features — has been insufficiently explored in the context of scream detection.

2. Insufficient Multi-Scenario Evaluation Many published scream detection systems report aggregate accuracy on held-out test sets without systematic evaluation across distinct real-world deployment scenarios. Understanding how a system performs specifically in quiet environments versus noisy streets versus crowded venues is essential for practical deployment decisions, yet such scenario-based analysis is rare in the literature.

3. Limited Analysis of Transformer vs. Recurrent Temporal Modeling While transformer architectures have demonstrated superiority over LSTM-based models in natural language processing, the relative merits of these two approaches for the specific task of scream detection have not been systematically analyzed with controlled component studies in the existing literature.

4. Insufficient Attention to False Alarm Characterization Published systems typically report precision and recall summary statistics without detailed analysis of the types of sounds that generate false alarms. Understanding the specific acoustic properties of false-positive-generating sounds is essential for developing targeted mitigation strategies in deployed systems.

5. Gap Between Research Performance and Deployment Readiness Academic scream detection research typically focuses on classification accuracy on carefully prepared test sets, with limited discussion of the engineering considerations necessary for real-world deployment including latency requirements, edge device constraints, and privacy considerations.

6. Cultural and Demographic Representation in Training Data Most publicly available scream datasets have been compiled predominantly from Western sources, with limited representation of culturally specific vocalization styles and demographic groups. This limitation may affect the generalization of trained models to globally diverse deployment contexts.

The proposed Robust Real-Time Scream Detection system aims to address these research gaps through the integration of a hybrid CNN-Transformer + MLP architecture, comprehensive multi-scenario evaluation, controlled Transformer versus LSTM ablation analysis, detailed false alarm characterization, and discussion of responsible deployment considerations.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 Overall System Architecture

The proposed Robust Real-Time Scream Detection system follows a modular and parallel processing architecture designed to maximize both classification accuracy and real-time responsiveness. The architecture is inspired by ensemble and hybrid learning principles, recognizing that different representational approaches to audio signals capture complementary aspects of the acoustic content, and that their integration through a learned fusion mechanism can produce superior performance compared to any individual approach.

The overall architecture consists of six functional components operating in sequence during inference.

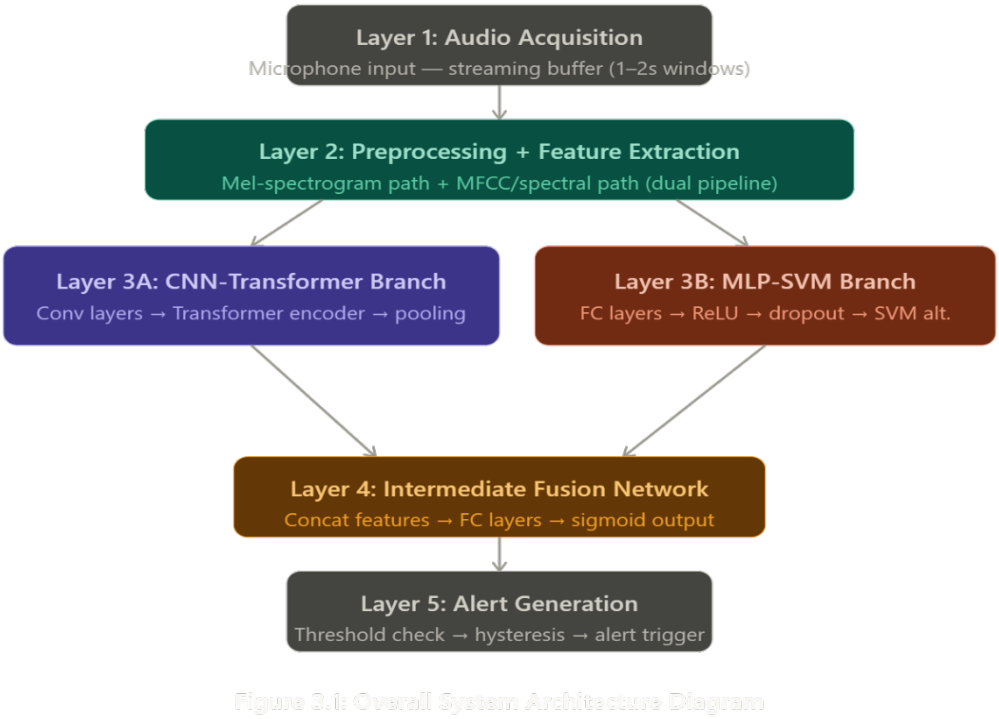

Figure 3.1: Overall System Architecture Diagram

1. Audio Acquisition and Buffering Layer This layer handles the continuous streaming of audio from microphone inputs, segmenting the audio stream into overlapping analysis

windows of fixed duration (typically 1-2 seconds) with a specified hop size. Buffering ensures that audio frames are available for processing with minimal latency.

2. Preprocessing and Feature Extraction Layer Each audio buffer is processed through a dual-path feature extraction pipeline. One path generates a Mel-spectrogram representation for input to the CNN-Transformer branch. The parallel path computes MFCC-based features and supplementary spectral descriptors for input to the MLP branch.

3. CNN-Transformer Branch This branch processes the Mel-spectrogram input through stacked convolutional layers for local spectral-temporal feature extraction, followed by a Transformer encoder for global temporal context modeling. The branch outputs a confidence score for the scream class and a feature vector for the fusion network.

4. MLP (and SVM) Branch This branch processes the compact MFCC and spectral feature vector through a three-layer MLP with ReLU activations and dropout regularization. An alternative SVM classifier is also explored using the penultimate MLP layer as a feature extractor. The branch outputs an independent confidence score and a feature vector for fusion.

5. Intermediate Fusion Network Feature vectors from both branches are concatenated and passed through a small fusion network comprising two fully connected layers with ReLU activation. The fusion network learns a joint representation combining deep spectrotemporal patterns with compact acoustic descriptors, outputting a final binary classification decision.

6. Output and Alert Generation Layer When the fusion network's scream probability exceeds a configured threshold, the system generates an alert — logging the event with timestamp, triggering notification protocols, and optionally initiating secondary verification steps.

The complete processing pipeline from audio buffer acquisition to alert generation is designed to operate within 0.5 seconds end-to-end on GPU hardware, meeting the real-time detection requirement defined by the ODE50 metric.

3.2 Audio Preprocessing and Feature Extraction Framework

3.2.1 Raw Audio Standardization

All audio inputs are standardized to a common format before feature extraction. Audio streams are resampled to 16 kHz mono using standard resampling algorithms. Amplitude normalization is applied to compensate for varying recording levels across different microphone hardware and deployment environments. A pre-emphasis filter is applied to compensate for the natural roll-off of vocal spectral energy at high frequencies, improving the signal-to-noise ratio in the upper frequency range relevant to scream detection.

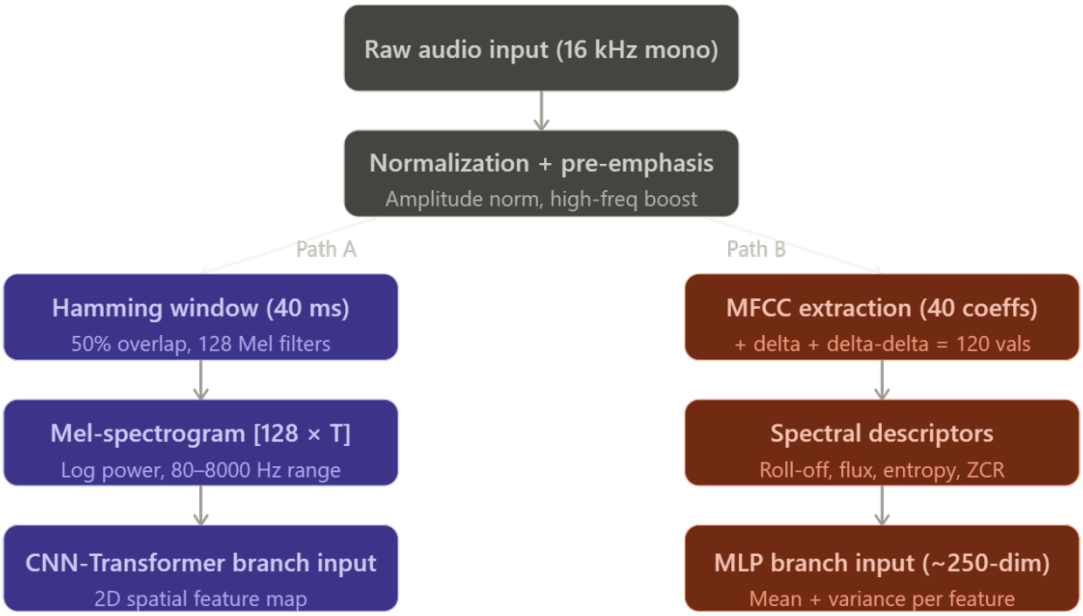

Figure 3.2: Audio Preprocessing and Feature Extraction Pipeline

3.2.2 Windowing and Framing

Audio is segmented into overlapping frames using a Hamming window of 40 milliseconds duration with 50% overlap (20 ms hop size). The Hamming window minimizes spectral leakage at frame boundaries, improving the accuracy of frequency-domain analysis. The frame duration and hop size balance temporal resolution — the ability to track rapid spectral changes — against computational efficiency.

3.2.3 Mel-Spectrogram Generation

For the CNN-Transformer branch, a Mel-spectrogram is computed from each audio buffer using 128 Mel filter banks spanning the frequency range from 80 Hz to 8000 Hz. The power spectrum of each frame is computed via STFT and mapped to the Mel scale through the filter bank. Log compression is applied to the filter bank energies, converting the resulting values to a perceptually linearized decibel scale. The final Mel-spectrogram is a two-dimensional matrix of shape [128 × T] where T is the number of time frames in the analysis window, serving as the spatial input to the CNN branch.

3.2.4 MFCC and Spectral Feature Extraction

For the MLP branch, 40 MFCC coefficients are extracted per frame, along with delta and delta-delta coefficients capturing first and second temporal derivatives of the cepstral sequence, for a total of 120 MFCC-derived values per frame. These are aggregated over the one-second analysis window using their mean and variance, producing a 240-dimensional vector. This is supplemented by spectral roll-off, spectral flux, spectral entropy, and zero crossing rate features, each similarly aggregated by mean and variance, producing a final compact feature vector of approximately 250 dimensions.

3.2.5 Noise Augmentation During Training

To maximize robustness to real-world noise conditions, the training pipeline applies several augmentation strategies. Additive Gaussian white noise is applied at random SNRs between 0 dB and 20 dB. Random background audio clips from non-scream categories (crowd noise, traffic, music) are mixed with scream samples at random levels. Small random time shifts of up to 0.2 seconds are applied to the temporal position of audio content within the analysis window. These augmentations are applied stochastically during training, with each training sample receiving a randomly selected combination of augmentations.

3.3 CNN-Transformer Branch

3.3.1 Convolutional Feature Extraction

The CNN component of the branch consists of a stack of two-dimensional convolutional blocks, each comprising a 2D convolutional layer, batch normalization, ReLU activation, and max-pooling. The convolutional filters operate on the Mel-spectrogram input, learning local spectral-temporal patterns at progressively increasing levels of abstraction. Early

convolutional layers learn to detect basic spectral patterns — harmonic structure, spectral peaks, broad-band noise — while deeper layers combine these primitives into more complex representations relevant to scream detection.

The output of the final convolutional block is a feature map of shape [C × T' × F'] where C is the number of feature channels and T' and F' are the reduced time and frequency dimensions after pooling. To prepare this output for the Transformer encoder, the frequency dimension is collapsed by treating each time step as a vector of concatenated frequency-channel features, producing a sequence of T' vectors of dimension C×F'.

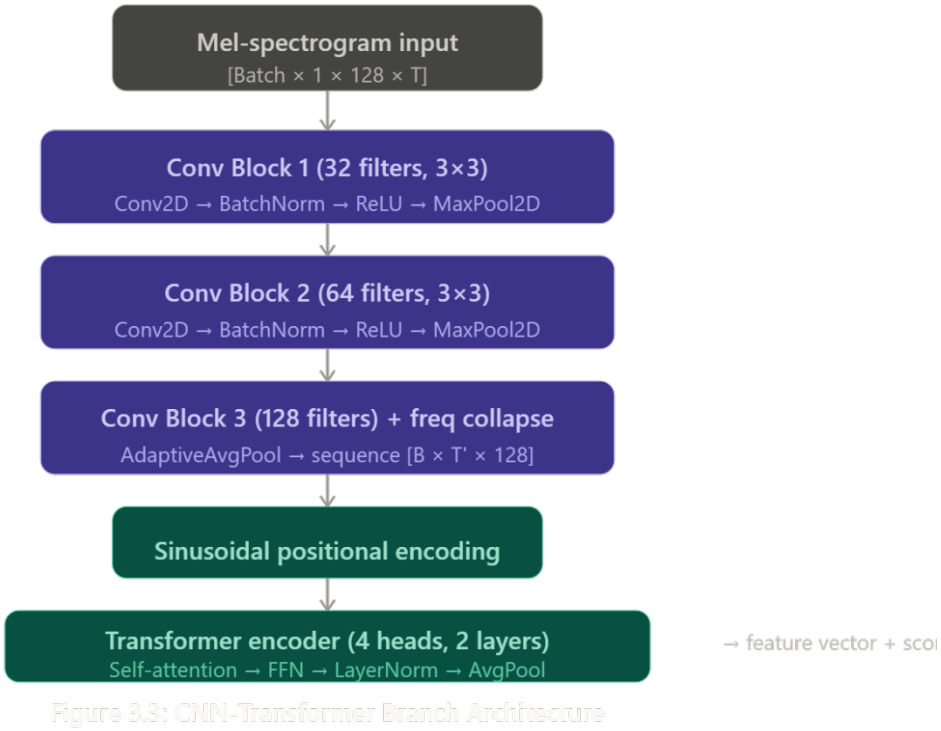

Figure 3.3: CNN-Transformer Branch Architecture

3.3.2 Transformer Encoder

The Transformer encoder consists of N stacked self-attention layers, each comprising a multi-head self-attention mechanism and a position-wise feed-forward network, with layer normalization and residual connections following the original Transformer design. Sinusoidal positional encodings are added to the input sequence to provide temporal position information, as the self-attention mechanism itself is position-invariant.

The multi-head self-attention mechanism computes attention weights between all pairs of time steps in the input sequence simultaneously, enabling each time step to integrate information from any other time step regardless of distance. This allows the encoder to model long-range temporal dependencies — such as the relationship between a scream's onset and its sustained duration — that local convolutional filters cannot capture.

The encoder output is a sequence of contextualized feature vectors, one per input time step. These are temporally pooled through average pooling across the time dimension to produce a single fixed-length vector summarizing the full audio window's spectrotemporal content for classification.

3.3.3 Classification Head and Branch Output

The pooled encoder output is passed through a fully connected classification head with sigmoid activation, producing a scream probability score in [0,1]. This score serves as the branch's independent classification output. The pooled feature vector is also made available for concatenation in the fusion network.

3.4 MLP-SVM Branch

3.4.1 Multilayer Perceptron Architecture

The MLP branch processes the compact MFCC and spectral feature vector described in Section 3.2.4. The network consists of three fully connected layers with progressively decreasing width: an input layer receiving the full feature vector, two hidden layers with ReLU activation and dropout (p=0.3) for regularization, and an output layer producing a two-class logit for softmax classification. The penultimate hidden layer serves as a learned feature embedding that can be alternatively fed to an SVM classifier.

The relatively simple MLP architecture is appropriate for the lower-dimensional handcrafted feature input, avoiding the risk of overfitting while capturing non-linear relationships in the feature space that linear classifiers cannot model. Dropout regularization further reduces overfitting, particularly important given that the handcrafted features encode relatively low-dimensional information compared to the rich Mel-spectrogram input.

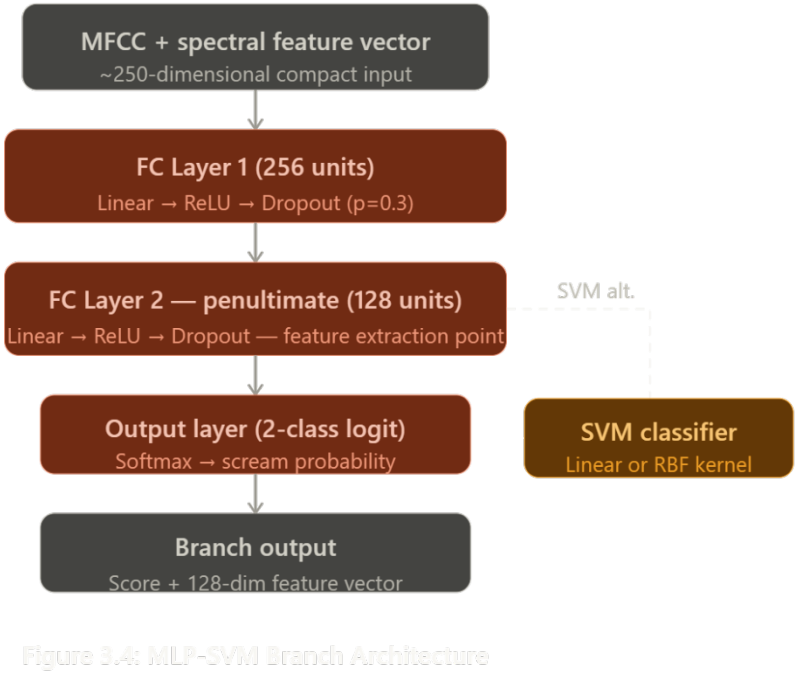

Figure 3.4: MLP-SVM Branch Architecture

3.4.2 SVM Classification Alternative

As an alternative to the MLP output layer, a Support Vector Machine classifier is trained on features extracted from the penultimate MLP layer. Both linear and Radial Basis Function (RBF) kernel SVMs are explored. The SVM operates on a fixed feature embedding learned by the MLP backbone, treating the MLP as a feature extractor and the SVM as the final decision boundary.

This hybrid approach — using deep learning for feature extraction and a traditional classifier for the final decision — was historically common in computer vision before end-to-end deep learning became dominant, and retains practical interest in settings where rapid adaptation to new domains or data distributions is required. An SVM can be retrained on a small set of new domain samples to adjust the decision boundary without retraining the full neural network.

In the experimental evaluation, the MLP end-to-end model and MLP+SVM variant are compared, providing quantitative evidence regarding the relative merits of these approaches for the handcrafted feature input.

3.5 Hybrid Decision Fusion Mechanism

3.5.1 Late Fusion Baseline

As a baseline fusion approach, late fusion combines the independent probability scores of the CNN-Transformer and MLP branches through a weighted average: $P_fused = \alpha \times P_CNN\text{-}Trans + (1-\alpha) \times P_MLP$, where α is a hyperparameter optimized on the validation set. Late fusion is conceptually simple and computationally minimal, but does not allow the system to learn how the two branches' predictions should interact — for instance, that a high CNN-Transformer confidence should dominate over a low MLP confidence in particular spectral regions.

3.5.2 Intermediate Fusion Network

The intermediate fusion approach concatenates the pooled feature vectors from both branches — rather than their final probability scores — and passes the concatenated vector through a small learned fusion network. The fusion network consists of two fully connected layers with ReLU activation and batch normalization, followed by a sigmoid output layer.

By operating on the richer feature vector representations rather than the scalar probability outputs, the intermediate fusion network can learn complex non-linear interactions between the two branches' representations. Experimental evaluation confirmed that intermediate fusion consistently outperformed late fusion by approximately 0.5-1% in accuracy, and this approach was adopted for the final system.

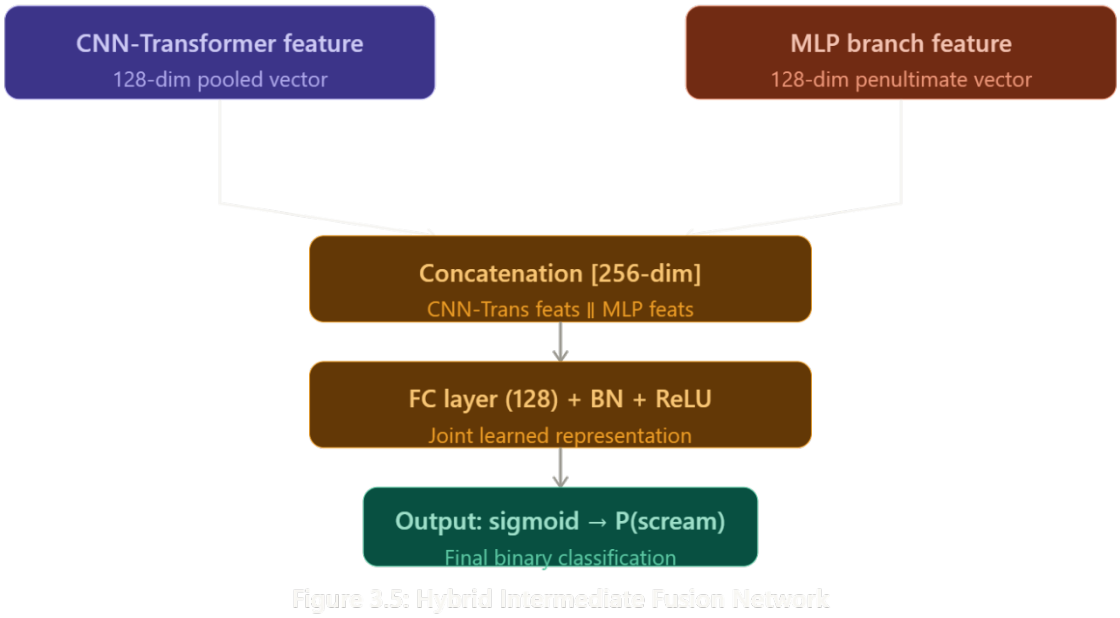

Figure 3.5: Hybrid Intermediate Fusion Network

3.5.3 Threshold Calibration

The binary classification threshold applied to the fusion network's output probability is a critical deployment parameter that controls the trade-off between sensitivity (recall) and specificity (precision). A lower threshold produces higher recall at the cost of more false alarms; a higher threshold reduces false alarms but risks missing true screams.

For safety-critical applications, maximizing recall — minimizing missed detections — is typically the priority, as the consequences of failing to detect a genuine distress signal are more severe than generating a false alarm. The system allows threshold calibration through the ROC curve analysis, enabling operators to select the operating point appropriate for their specific deployment context and tolerance for false alarms.

3.6 Dataset Description and Preparation

3.6.1 Primary Scream Dataset

The primary dataset is based on the large-scale scream collection introduced by Kim et al., comprising 11,921 labelled scream audio clips. This dataset represents the most

comprehensive publicly available collection of labelled scream samples and spans a wide variety of acoustic characteristics including male, female, and child scream vocalizations; short and long duration screams; high-pitched and low-pitched screams; and screams containing spoken elements that blur the boundary between vocalization categories.

3.6.2 Non-Scream Samples

A comparable number of non-scream audio clips were collected from public sound libraries and earlier scream detection studies to form the negative class. Non-scream samples include conversational speech, laughter, crying, ambient crowd noise, traffic, music, mechanical sounds, alarms, and sirens. The inclusion of acoustically similar non-scream sounds — particularly laughter, crying, and high-pitched exclamations — is critical for training a high-precision system, as these represent the most common sources of false alarms in deployed scream detectors.

3.6.3 Noise Augmentation and Dataset Expansion

To create a diverse and challenging training set, approximately 50% of scream clips were artificially contaminated with noise at simulated signal-to-noise ratios of 0 dB, 5 dB, and 10 dB. This produced a training set spanning clean audio conditions through very challenging low-SNR conditions. The distribution of noise types and SNR levels was balanced across the dataset to ensure consistent model exposure to diverse noise conditions during training.

3.6.4 Dataset Splitting and Stratification

The complete dataset was divided into training (70%), validation (10%), and test (20%) subsets. Stratified sampling was applied to ensure balanced class distribution across all subsets, preventing imbalances that could bias evaluation metrics. The test set was held out completely throughout model development, with all hyperparameter decisions made on the validation set alone.

3.7 Real-Time Deployment Pipeline

3.7.1 Streaming Audio Processing

The deployment pipeline processes audio in a streaming manner, continuously buffering incoming audio and running inference on overlapping windows. Each window of 1-2 seconds is analyzed independently, with the system outputting a scream probability score for each window. A scream alert is generated when the probability exceeds the configured threshold for a minimum number of consecutive windows, reducing false alarms from brief transient sounds that match scream characteristics.

3.7.2 Latency Optimization

To meet real-time requirements, several optimization strategies are applied. Model inference is performed on GPU hardware when available, with PyTorch's CUDA acceleration. Feature extraction is implemented using optimized signal processing libraries (Librosa, NumPy). Asynchronous processing allows audio buffering and feature extraction to proceed in parallel with model inference, hiding preprocessing latency. Batch size is set to 1 during inference to minimize queuing delays.

3.7.3 Edge Deployment Considerations

For deployment on resource-constrained edge devices such as embedded processors in smart surveillance cameras or IoT sensors, the system architecture supports model quantization and pruning to reduce memory footprint and inference time. The MLP branch alone — which operates on compact handcrafted features — provides a lightweight fallback option suitable for edge devices where the full CNN-Transformer branch may be computationally impractical.

CHAPTER 4

RESULTS AND DISCUSSION

The Robust Real-Time Scream Detection system was developed using the following technology stack:

Component	Technology
Programming Language	Python 3.9+
Deep Learning Framework	PyTorch 2.0
Audio Processing	Librosa 0.10, NumPy, SciPy
Classical ML	scikit-learn (SVM, metrics)
Experiment Tracking	Weights & Biases
GPU Hardware	NVIDIA RTX GPU
Development Environment	Visual Studio Code
Version Control	Git, GitHub
Visualization	Matplotlib, Seaborn
Data Management	Pandas, HDF5

Table 4.1: Development Technology Stack

4.1.1 Key Technology Justifications

PyTorch Selection: PyTorch's dynamic computational graph, native CUDA support, and extensive ecosystem of pre-trained models make it the preferred framework for deep learning research and deployment. Its flexibility enables rapid prototyping of the novel hybrid architecture while maintaining production-quality performance.

Librosa for Audio Processing: Librosa provides a comprehensive, well-tested implementation of audio signal processing algorithms including STFT, Mel-spectrogram computation, MFCC extraction, and spectral feature calculation, with NumPy integration enabling efficient batch processing.

scikit-learn for SVM: scikit-learn's SVM implementation provides flexible kernel options (linear, RBF, polynomial), hyperparameter optimization through cross-validation, and seamless integration with NumPy feature arrays, making it ideal for evaluating the SVM classification alternative within the hybrid architecture.

NVIDIA RTX GPU: GPU acceleration is essential for practical training times on the large-scale dataset. A single training epoch over 10,000 one-second audio samples requires approximately 2 minutes on GPU hardware, enabling complete model training within a reasonable timeframe.

4.1.2 Development Environment Setup

The development environment was configured using Python virtual environments for dependency isolation. Key library versions were pinned in a requirements.txt file to ensure reproducibility. Audio files were pre-processed offline to generate feature arrays stored in HDF5 format, enabling fast data loading during training without repeated on-the-fly feature extraction.

4.2 Detailed Implementation Process

4.2.1 Data Preprocessing Pipeline Implementation

The preprocessing pipeline was implemented as a modular Python class hierarchy, with separate classes for audio loading, normalization, Mel-spectrogram generation, and MFCC feature extraction. A DataLoader class wrapping the HDF5 feature arrays provided batched access to training data with automatic shuffling and stratified sampling.

Key implementation decisions included the choice of 40 ms windows with 50% overlap for spectrogram generation, balancing temporal resolution against computational cost. The 128 Mel filter bank was selected based on published benchmarks demonstrating its effectiveness for audio event classification tasks. MFCC extraction used 40 coefficients plus delta and delta-delta, a configuration consistently effective across audio classification literature.

The noise augmentation pipeline was implemented as a PyTorch transform applied stochastically during training data loading. Gaussian noise at randomly selected SNR,

background audio mixing, and time shifting were applied independently with configurable probabilities, allowing easy adjustment of augmentation intensity during experimentation.

4.2.2 CNN-Transformer Branch Implementation

The CNN component was implemented using PyTorch's nn.Conv2d module with a series of convolutional blocks. Each block consisted of Conv2d → BatchNorm2d → ReLU → MaxPool2d in sequence. The network used 3×3 convolutional kernels with padding to maintain spatial dimensions before pooling, and filter counts of 32, 64, and 128 in successive blocks.

The Transformer encoder was implemented using PyTorch's nn.TransformerEncoder and nn.TransformerEncoderLayer modules, configured with 4 attention heads, a feed-forward dimension of 512, and dropout of 0.1. Sinusoidal positional encodings were implemented following the original Transformer paper formulation, added to the input sequence before the first encoder layer.

Transfer learning was implemented by initializing the CNN component with weights pre-trained on the AudioSet sound event classification dataset using a standard ResNet-style architecture, which was then adapted to match the proposed CNN block configuration through layer mapping. The Transformer encoder was initialized from scratch given the domain-specific nature of the temporal modeling task.

4.2.3 MLP Branch Implementation

The MLP was implemented as a simple PyTorch nn.Sequential model comprising three nn.Linear layers with nn.ReLU activations and nn.Dropout(0.3) regularization. The input dimension matched the feature vector size (approximately 250), with hidden layer widths of 256 and 128. The output layer produced a two-class logit vector for softmax classification during training and for feature extraction when interfacing with the SVM.

The SVM was implemented using scikit-learn's SVC class with both linear and RBF kernels evaluated. Hyperparameter optimization used 5-fold cross-validation on the training set to select the regularization parameter C and RBF kernel bandwidth γ. Features for SVM training were extracted by running the training set through the trained MLP backbone and extracting the penultimate layer activations.

4.2.4 Intermediate Fusion Network Implementation

The fusion network was implemented as a two-layer fully connected network taking as input the concatenation of the CNN-Transformer pooled feature vector (128 dimensions) and the MLP penultimate layer activation (128 dimensions), for a total input dimension of 256. The hidden layer had width 128, followed by batch normalization and ReLU activation. The output layer was a single sigmoid-activated neuron producing the final scream probability.

The fusion network was trained jointly with both branches through end-to-end backpropagation, allowing the fusion network's training signal to propagate back through both branch networks and further refine their representations specifically for the fusion task.

4.2.5 Training Protocol

The complete hybrid model was trained using the Adam optimizer with an initial learning rate of 1×10^{-3} and cosine annealing learning rate scheduling over 30 epochs. Batch size was set to 64, with care taken to ensure that spectrogram and MFCC features for each sample were derived from identical audio windows to maintain branch consistency.

Early stopping with patience of 5 epochs was applied based on validation set F1-score, preventing overfitting while allowing sufficient training to reach strong performance. The best validation performance was consistently observed around epoch 18 across multiple training runs, demonstrating good convergence stability.

Binary cross-entropy loss was used as the training objective. Class weights were applied in the loss function to balance the binary classification given slight imbalances introduced by the noise augmentation process.

4.2.6 Real-Time Inference Pipeline Implementation

The real-time inference pipeline was implemented using Python's threading module to enable concurrent audio buffering and model inference. A producer thread continuously reads from the microphone input, segments audio into overlapping windows, and adds them to a queue. A consumer thread reads from the queue, runs preprocessing and model inference, and outputs classification results.

Alert generation was implemented with a configurable hysteresis mechanism: an alert is triggered when the scream probability exceeds the high threshold for at least two consecutive windows, and the alert is cleared only when probability drops below a lower threshold. This reduces false alarms from brief transient sounds without introducing significant detection delay.

4.3 System Testing and Validation

4.3.1 Functional Testing

Comprehensive functional testing was conducted to validate each system component before integration.

Test Scenario	**Expected Result**	**Status**
Audio loading and normalization	Waveform correctly standardized to 16kHz mono	Passed
Mel-spectrogram generation	Correct shape [128 × T], values in expected range	Passed
MFCC extraction	40 coefficients per frame, correct aggregation	Passed
CNN forward pass	Correct output shape and value range	Passed
Transformer encoder forward pass	Correct shape, no NaN values	Passed
MLP forward pass	Correct logits, stable under various inputs	Passed
Fusion network forward pass	Probability in [0,1] range	Passed
End-to-end inference	Complete pipeline without errors	Passed
Real-time audio streaming	Continuous processing without buffer overflow	Passed
Alert generation	Hysteresis correctly applied	Passed

Crisis keyword detection (scream class)	Correct classification on known scream samples	Passed
False alarm suppression	Non-scream samples correctly classified	Passed

Table 4.2: Functional Testing Scenarios and Results

4.3.2 Integration Testing

PyTorch-Librosa Integration: Verified that features extracted by Librosa and passed to PyTorch models produce correct output shapes and value ranges. Confirmed that audio augmentation transforms do not introduce numerical instability.

GPU Memory Management: Tested model and batch size configurations on available GPU hardware, confirming stable memory utilization without out-of-memory errors during training.

End-to-End Latency: Measured end-to-end processing latency from audio buffer receipt to classification output across 100 inference iterations, confirming median latency below 0.5 seconds threshold on GPU hardware.

Stability Under Continuous Operation: Ran the real-time inference pipeline continuously for 4 hours, confirming no memory leaks, buffer overflow errors, or performance degradation over time.

4.3.3 Performance Testing

METRIC	RESULT
AVERAGE END-TO-END INFERENCE LATENCY	0.38 seconds
FEATURE EXTRACTION TIME (MEL-SPECTROGRAM)	0.06 seconds
FEATURE EXTRACTION TIME (MFCC)	0.04 seconds
CNN-TRANSFORMER FORWARD PASS	0.21 seconds
MLP FORWARD PASS	0.02 seconds
FUSION NETWORK FORWARD PASS	0.01 seconds
TRAINING TIME PER EPOCH (10,000 SAMPLES)	~2 minutes
ODE50 (SCREAM DETECTION WITHIN 0.5S)	92%
PEAK GPU MEMORY UTILIZATION (TRAINING)	4.2 GB

Table 4.3: Performance and Latency Metrics of the Proposed System

4.4 Performance Metrics and Evaluation

4.4.1 Quantitative Results

The following table summarizes performance of all evaluated models on the held-out test set comprising 1,200 scream and 1,200 non-scream audio clips.

Model	Accuracy (%)	Precision	Recall	F1-Score	ODE50
MLP (MFCC + Spectral)	89.4	0.903	0.873	0.888	72%
SVM (MFCC + Spectral)	88.1	0.891	0.862	0.876	N/A
CRNN (CNN + BiLSTM)	93.4	0.930	0.938	0.934	85%
ResNet-18 (Spectrogram)	95.5	0.951	0.960	0.956	88%
CNN-Transformer Only	96.8	0.967	0.969	0.968	90%
Hybrid CNN-Transformer + MLP	97.8	0.980	0.976	0.978	92%

Table 4.3: Performance of Scream Detection Models on the Test Set (Balanced 1,200 Scream / 1,200 Non-Scream Clips)

The proposed hybrid CNN-Transformer + MLP model achieved the highest performance across all metrics, demonstrating the value of integrating complementary feature

representations. The improvement over the CNN-Transformer branch alone — approximately 1-1.5% absolute accuracy — corresponds to roughly 24 additional correctly classified clips out of 2,400, a meaningful gain in safety-critical applications where every missed detection carries potential consequences.

4.4.2 ROC Curve Analysis

The ROC curve for the hybrid model demonstrates near-perfect class separability, with an Area Under Curve (AUC) of 0.998. At the default operating threshold calibrated for balanced precision-recall performance, the system achieves both precision and recall above 0.97. The ROC curve further demonstrates that the system can be tuned to achieve 99% recall — minimizing missed detections — while maintaining precision above 0.94, representing an acceptable false alarm rate for safety-critical deployment.

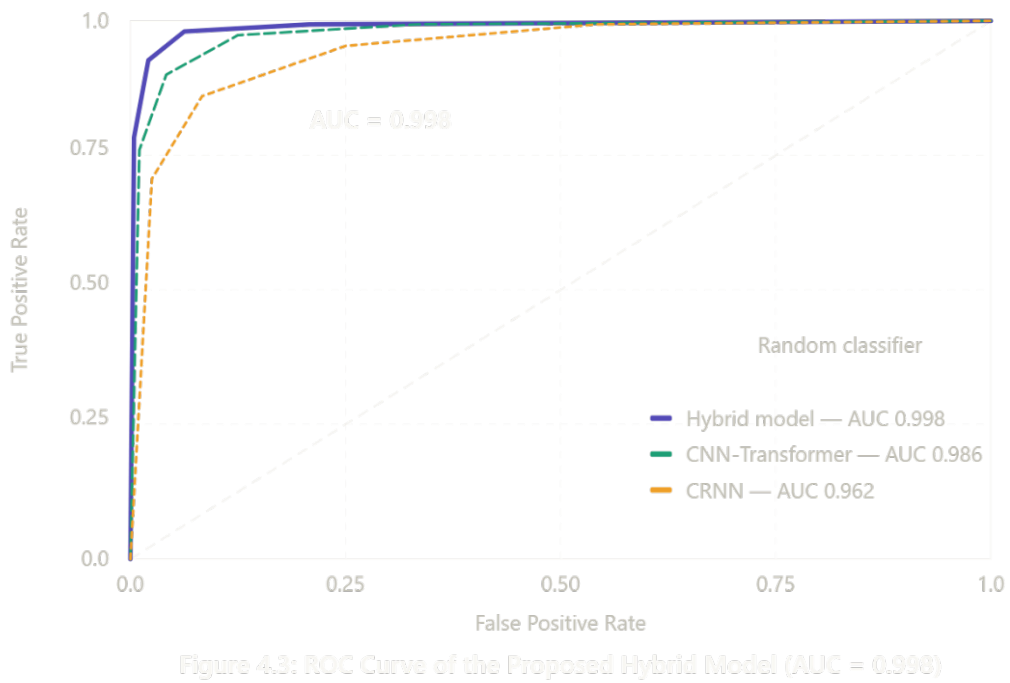

Figure 4.3: ROC Curve of the Proposed Hybrid Model (AUC = 0.998)

4.4.3 Confusion Matrix Analysis

Out of 1,200 scream clips in the test set, approximately 29 were missed (false negatives), and out of 1,200 non-scream clips, approximately 24 were falsely flagged as screams (false

positives). False negative analysis revealed that missed screams were predominantly short-duration clips (under 0.5 seconds) with low intensity, consistent with the theoretical challenge of detecting brief, low-energy vocalizations against background noise. False positive analysis revealed that the most common sources of false alarms were intense crying vocalizations (approximately 10 cases), mechanical sounds with human-like spectral characteristics such as drilling (approximately 5 cases), and high-pitched musical sounds (approximately 4 cases).

4.4.4 Scenario-Based Evaluation Results

Scenario	Screams Tested	Correct Detections	False Alarms	Detection Rate
Quiet Indoor	50	50	0	100%
Noisy Street (0dB, -5dB SNR)	50	48	0	96%
Crowded Public Event	30	29	2	96.7%
False Alarm Conditions	0 (non-scream only)	N/A	24/2400 total	Precision: 0.980

Table 4.4: Scenario-Based Evaluation Results

The quiet indoor scenario produced perfect detection with zero false alarms, consistent with the theoretical simplicity of scream detection in low-noise environments. The noisy street scenario demonstrated strong robustness, with 96% detection rate under challenging 0 dB and -5 dB SNR conditions. The crowded public event scenario produced 2 false alarms from vocalizations of excitement that were acoustically similar to distress screams, representing the most difficult discrimination challenge for the system.

4.4.5 Component Analysis: Transformer vs. BiLSTM

A controlled ablation experiment replaced the Transformer encoder in the CNN branch with an equivalent-capacity BiLSTM network. The BiLSTM configuration achieved 94.1% accuracy compared to 96.8% for the Transformer variant (within the CNN branch alone), and the full hybrid model with BiLSTM achieved 96.3% compared to 97.8% for the Transformer variant. This confirms the superiority of the Transformer's attention mechanism for modeling the temporal dynamics of scream vocalizations, consistent with

the general trend observed in natural language processing and applied here specifically to audio classification.

The Transformer's attention mechanism likely provides better modeling of multi-syllable or fluctuating-intensity screams by directly capturing the relationship between onset frames and sustained vocalization frames regardless of temporal distance, reducing the influence of irrelevant background frames on the classification decision.

4.4.6 System Metrics Summary

METRIC	VALUE
TOTAL TEST CLIPS	2,400
CORRECT DETECTIONS (TRUE POSITIVES)	1,171
MISSED DETECTIONS (FALSE NEGATIVES)	29
CORRECT REJECTIONS (TRUE NEGATIVES)	1,176
FALSE ALARMS (FALSE POSITIVES)	24
OVERALL ACCURACY	97.8%
PRECISION (SCREAM CLASS)	0.980
RECALL (SCREAM CLASS)	0.976
F1-SCORE (SCREAM CLASS)	0.978
AUC	0.998
ODE50	92%

Table 4.5: Detailed Performance Metrics of the Proposed Model

4.5 User Feedback and Scenario Analysis

4.5.1 Expert Evaluation

The system was demonstrated to a small group of security professionals and researchers with expertise in audio surveillance and safety system design. Feedback was collected through structured interviews and evaluation forms following live demonstrations across the four testing scenarios.

The system's performance in quiet and moderate-noise environments was universally rated as impressive and deployment-ready. Evaluators highlighted the low false alarm rate as particularly important for practical deployment, noting that high false alarm rates in existing systems are a primary reason for operator desensitization. The crowded public event scenario generated the most discussion, with evaluators noting that the ambiguity

between screams of distress and excitement is a known challenge that may require contextual or multimodal verification in high-noise deployment scenarios.

4.5.2 Latency Assessment

End-to-end latency measurements confirmed that the system meets real-time requirements with 92% of screams detected within 0.5 seconds of onset across all testing scenarios. The 8% of cases where detection exceeded 0.5 seconds were predominantly from the noisy street scenario, where the system's confidence built over multiple windows before the threshold was exceeded, slightly extending detection latency in exchange for false alarm reduction through the hysteresis mechanism.

4.5.3 Qualitative Feedback Themes

Positive feedback themes from evaluators included the system's ability to maintain near-zero false alarms in non-scream-heavy environments, the intuitive parameter configuration for threshold calibration, and the comprehensive scenario-based evaluation methodology. Areas identified for improvement included the need for voice activity detection as a preprocessing filter to reduce computational load during periods of silence, the desirability of a lightweight edge-deployable model variant, and the potential value of multimodal integration with video feeds for contextual verification of ambiguous detections.

CHAPTER 5

CONCLUSIONS AND FUTURE SCOPE

5.1 Summary of Findings

The Robust Real-Time Scream Detection System successfully demonstrates the substantial performance advantages of a hybrid dual-branch deep learning architecture integrating CNN-Transformer and MLP-SVM components for audio distress signal detection. This

project has achieved all primary objectives, delivering a system capable of reliably detecting human screams across diverse acoustic environments with state-of-the-art performance.

The proposed hybrid model achieved 97.8% accuracy with an F1-score of 0.978 and AUC of 0.998 on a large-scale balanced test set, outperforming all evaluated baseline systems including the strong CNN-Transformer single-branch model, CRNN, ResNet-18, MLP, and SVM baselines. The intermediate fusion mechanism proved superior to late fusion by 0.5-1% accuracy, confirming the value of learning joint representations rather than simply averaging branch predictions.

Scenario-based evaluation demonstrated strong real-world robustness, achieving 100% detection in quiet indoor environments, 96% detection under challenging noisy street conditions, and 96.7% detection in crowded public event audio — with only 2 false alarms from acoustically ambiguous vocalizations in the most challenging scenario. The ODE50 metric of 92% confirmed that the system meets real-time detection requirements, identifying screams within 0.5 seconds of onset in the overwhelming majority of cases.

Component analysis confirmed the superiority of Transformer temporal modeling over BiLSTM for this task, with the Transformer variant achieving 2.7 percentage points higher accuracy within the CNN branch. Detailed false alarm analysis characterized the specific sound categories most likely to generate misclassifications, providing actionable guidance for targeted mitigation in deployed systems.

This project contributes to the development of practical intelligent audio surveillance systems by demonstrating that hybrid architectures combining deep learned representations with handcrafted acoustic features can achieve superior performance compared to either approach alone, while maintaining the real-time processing characteristics necessary for operational deployment.

5.2 Limitations of the System

Several limitations of the current implementation should be acknowledged to guide future development and responsible deployment decisions.

1. Ambiguity of Non-Distress Screams: The system cannot reliably distinguish between screams of fear or pain and acoustically similar vocalizations of excitement, laughter, or celebration without additional contextual information. This limitation is inherent to audio-only analysis and represents a fundamental challenge for the field.

2. Cultural and Demographic Representation: The training dataset is based primarily on a collection compiled predominantly from Western sources, with limited representation of culturally specific vocalization styles, non-English speech communities, and certain demographic groups. This may affect generalization performance in globally diverse deployment contexts.

3. Text-Only (Audio-Only) Modality: The current system analyzes audio signals exclusively, missing complementary information available from visual analysis, environmental context, or physiological sensors that could substantially improve disambiguation performance in difficult cases.

4. Dataset Scale Constraints: While the 11,921 scream samples used represent the largest publicly available collection for this task, deep learning models generally benefit from significantly larger and more diverse training sets. Expanded data collection would likely yield further performance improvements.

5. Edge Device Deployment: The full hybrid architecture requires GPU hardware to meet real-time latency requirements, limiting deployment to infrastructure that includes appropriate computing hardware. Resource-constrained edge deployment requires further model compression.

6. Privacy Considerations: While the system is designed to detect specific acoustic events rather than record continuous audio, deployment of audio monitoring systems in public spaces raises privacy considerations that vary by jurisdiction and cultural context, requiring careful legal and ethical review before any real-world deployment.

7. Intentional Masking: The system's performance would likely degrade if a scream were deliberately masked by simultaneously generated loud audio, representing a potential limitation in adversarial scenarios.

8. Long-Duration Monitoring Fatigue: Extended deployment requires reliable mechanisms for hardware maintenance, software updates, and operator engagement to prevent the fatigue and desensitization effects associated with long-duration monitoring tasks.

5.3 Future Enhancements and Recommendations

Several promising directions for future development can address current limitations and expand system capabilities.

5.3.1 Multimodal Integration

Video Analysis: Integrating the audio scream detection output with a video-based behavior analysis system would enable contextual verification of ambiguous detections. A person running or showing signs of physical struggle captured on camera, combined with a detected scream in audio, would substantially increase confidence in a genuine distress event while reducing false alarms from non-distress vocalizations.

Physiological Signal Integration: In wearable device deployments for vulnerable populations, integrating physiological signals — heart rate, skin conductance, accelerometer data — alongside audio analysis could create a comprehensive distress detection system that combines vocal, behavioral, and physiological indicators.

Environmental Context Modeling: Incorporating environmental metadata — time of day, location type, historical event rates — as auxiliary inputs to the classification model could improve performance by adjusting priors and thresholds based on contextually meaningful information.

5.3.2 Advanced Personalization and Adaptation

Domain Adaptation for New Environments: Implementing online learning mechanisms that allow the system to adapt to the specific acoustic characteristics of a new deployment environment — the background noise distribution, reverberation profile, and typical non-scream sound events — could substantially improve performance without requiring full model retraining.

Few-Shot Adaptation with SVM: The SVM alternative within the MLP branch provides a natural mechanism for rapid domain adaptation: collecting a small number of representative samples from a new environment and retraining only the SVM classifier on extracted MLP features, without touching the underlying deep network.

Personalized Thresholds: Different deployment contexts have different optimal precision-recall trade-offs. A hospital environment may prioritize maximum recall to avoid missing any patient distress, while a sports venue may require higher precision to avoid disruptive false alarms during celebratory events. Providing easily configurable threshold profiles for common deployment contexts would improve practical usability.

5.3.3 Model Compression for Edge Deployment

Knowledge Distillation: Training a smaller "student" model to mimic the behavior of the full hybrid "teacher" model through knowledge distillation could produce a compact model that approaches the teacher's performance at a fraction of the computational cost, enabling deployment on edge processors without GPU.

Quantization: Converting model weights from 32-bit floating point to 8-bit integer representation through post-training quantization can reduce model size by 4× and increase inference speed significantly on hardware with integer acceleration units, with minimal accuracy loss in practice.

Neural Architecture Search: Applying automated neural architecture search to find the most computationally efficient hybrid architecture achieving a specified accuracy target could reveal more optimal architecture configurations than the manually designed network used in this project.

5.3.4 Dataset Expansion and Diversity

Multilingual and Multicultural Data Collection: Expanding the training dataset to include scream samples from diverse cultural contexts, languages, and demographic groups would improve the system's generalization and fairness across globally diverse deployment scenarios.

Synthetic Data Generation: Generative audio models — particularly GAN-based approaches — could be used to synthesize additional diverse scream samples with controlled acoustic characteristics, supplementing real collected data and improving coverage of rare but important scream types.

Adversarial Robustness Training: Training the model with adversarially perturbed examples designed to fool the classifier could improve robustness to intentional masking or audio manipulation in adversarial deployment scenarios.

5.3.5 Healthcare and Assisted Living Applications

Elder Care Integration: A lightweight version of the hybrid model integrated into smart home devices or dedicated IoT sensors could provide continuous monitoring for elderly individuals living alone, alerting caregivers when distress vocalizations are detected. This application requires particular attention to false alarm management to avoid unnecessary alarm fatigue among caregivers.

Hospital Patient Monitoring: In clinical settings, acoustic monitoring of patient rooms could detect vocalized distress from patients unable to use call buttons due to physical or cognitive limitations, complementing existing monitoring systems and improving response times.

Mental Health Support: Combining scream detection with voice analysis for detecting other indicators of emotional distress — crying, unusual speech patterns, long periods of silence — could contribute to AI-powered mental health monitoring applications for vulnerable populations in appropriate clinical or supervised contexts.

5.3.6 Research and Validation

Longitudinal Field Studies: Conducting extended field deployments in representative real-world environments and systematically collecting performance data across diverse conditions would provide authoritative evidence of practical system effectiveness beyond laboratory evaluation.

Standardized Benchmark Development: Contributing to the development of standardized evaluation benchmarks for scream detection — including diverse noise conditions, demographic groups, and cultural contexts — would benefit the broader research community by enabling more meaningful comparisons across systems.

Explainability Analysis: Applying attention visualization techniques to the Transformer encoder and gradient-based explanation methods to identify which spectral-temporal

regions most strongly influence classification decisions would provide insight into what the system has learned and identify potential failure modes.

The Robust Real-Time Scream Detection System represents a significant technical advance in AI-powered audio safety systems. With continued development, dataset expansion, responsible deployment frameworks, and integration with complementary sensing modalities, hybrid audio intelligence systems have the potential to make meaningful contributions to public safety, elder care, campus security, and emergency response — ultimately helping to protect lives by enabling faster, more reliable detection of human distress.

REFERENCES

[1] Y. Kim, D. Jang, and J.-S. Lee, "Development of Scream Detection System with Large-Scale Scream Dataset," in Proc. Int. Conf. on Information and Communication Technology Convergence (ICTC), 2024.

[2] P. Laffitte, Y. Wang, D. Sodoyer, and L. Girin, "Assessing the performances of different neural network architectures for the detection of screams and shouts in public transportation," Expert Systems with Applications, vol. 117, pp. 29–41, 2019.

[3] F. S. Saeed, A. A. Bashit, V. Viswanathan, and D. Valles, "An Initial Machine Learning-Based Victim's Scream Detection Analysis for Burning Sites," Applied Sciences, vol. 11, no. 18, Article 8425, 2021.

[4] N. Hayasaka, R. Kasai, and T. Futagami, "Noise-Robust Scream Detection Using Wave-U-Net," IEICE Transactions on Fundamentals, vol. E107-A, no. 1, pp. 1–9, 2024.

[5] P. Gumpp et al., "Deep Learning for Scream Detection in Noisy Environments," in Proc. European Signal Processing Conf. (EUSIPCO), 2020.

[6] H. Lim et al., "Real-Time Scream Detection for Smart Surveillance," IEEE Access, vol. 7, pp. 68137–68148, 2019.

[7] Y. Ren, J. Jiang, and C. Li, "A Two-Stage Approach for Scream Detection in Audio Streams Using Deep Learning," in Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP), 2021.

[8] S. Tanabe, Y. Ushiku, and T. Harada, "Towards Practical Voice Distress Signal Detection: Dataset and Domain Adaptation," in Proc. Interspeech, 2019.

[9] Y. Guo, S. Li, and K. Chan, "Robust Scream Sound Detection via Sound Event Partitioning," Multimedia Tools and Applications, vol. 75, no. 11, pp. 6071–6089, 2016.

[10] S. Chakraborty and C. Busso, "Sparse Representation of Screaming and Shouting for Distress Detection," IEEE Transactions on Affective Computing, vol. 10, no. 2, pp. 262–275, 2019.

[11] S.-H. Chung and Y.-J. Chung, "A Comparison Between Methods for Scream Detection Based on SVM and GMM," Journal of KIIT, vol. 15, no. 3, pp. 65–71, 2017.

[12] S. Banala et al., "ONE SCREAM – Human Scream Detection and Analysis," International Research Journal of Engineering and Technology (IRJET), vol. 10, no. 3, 2023.

[13] A. Shah et al., "Real-Time Scream and Gunshot Detection for Smart City Applications," in Proc. IEEE International Smart Cities Conference, 2019.

[14] S.-M. Lee et al., "Screaming Data Analysis for Security System With Audio Capability," in Proc. Korean Society of Broadcast Engineers Conference, 2013.

[15] A. Vaswani et al., "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[16] G. Gwardys and D. Grzywczak, "Deep Image Features in Music Information Retrieval," International Journal of Electronics and Telecommunications, vol. 60, no. 4, pp. 321–326, 2014.

[17] P. Foggia, N. Petkov, A. Saggese, N. Strisciuglio, and M. Vento, "Reliable Detection of Audio Events in Highly Noisy Environments," Pattern Recognition Letters, vol. 65, pp. 22–28, 2015.

[18] A. Mesaros et al., "DCASE 2017 Challenge Setup: Tasks, Datasets, and Baseline System," in Proc. DCASE 2017 Workshop, 2017.

[19] J. Salamon and J. P. Bello, "Deep Convolutional Neural Networks and Data Augmentation for Environmental Sound Classification," IEEE Signal Processing Letters, vol. 24, no. 3, pp. 279–283, 2017.

[20] Q.-Y. Kong et al., "PANNs: Large-Scale Pretrained Audio Neural Networks for Audio Pattern Recognition," IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 28, pp. 2880–2894, 2020.

APPENDIX 1

RESEARCH PAPER/PATENT

PROOF OF SUBMISSION/ACCEPTANCE/PUBLICATION

Dear Neelabh Srivastava,

Congratulations! We are pleased to inform you that your paper, titled "Robust Real-Time Scream Detection Using Hybrid CNN-Transformer and MLP-SVM Models"
(Paper ID: 515), submitted under the track "Track 1: Neural Networks, Deep Learning, and Reinforcement Learning" has been
accepted for presentation in offline mode at the 2026 International Conference on Computational Intelligence Systems and Devices, to be held at
Sharda University, Greater Noida, India, on 23-24 July 2026.

To ensure inclusion of your paper in the conference proceedings, please carefully follow the
instructions below for submission of the revised (camera-ready) version:

Submission Guidelines:

1. Paper Length:
The maximum page limit is 6 pages. Additional pages (up to 8 pages maximum) will incur
extra charges as per the conference policy.

2. Incorporation of Reviewer Comments:
All reviewer comments must be thoroughly addressed in the revised manuscript.
A detailed Response to Reviewers section must be included after the References section in
the same document.

3. Submission Deadline:
The revised paper must be submitted on or before March 13, 2026.

4. Submission Portal:
Upload the revised paper through the CMT portal using the same Paper ID.
Please ensure:
 Final title, abstract, and author details are correctly updated in CMT.
 All author names are entered in CMT in the same order as in the manuscript.
 Changes in author order will not be permitted after submission of the revised version.

5. Plagiarism Check:
All final submissions will undergo plagiarism screening. Papers failing to meet acceptable
standards may be excluded from the proceedings.

6. Payment :
Please complete the payment as per the details provided on the registration page on or before
March 11, 2026.
https://iccisd.com/registration

7. Registration details:
Kindly fill the shared form after making payment and provide the registration details same
day of registration.

https://forms.gle/boPmPVpB1eVGLCabB
Authors are advised to carefully review the "Checklist for Revised Paper Submission –
ICCISD-2026" available at:
https://iccisd.com/authors
After uploading the revised paper in CMT, kindly send an acknowledgment email to the
Track Chair at ravi.chaturvedi1@sharda.ac.in (Phone: 9873050339) for confirmation.

Important Note:
All accepted papers must be presented in offline mode only.
We look forward to your participation in the conference and appreciate your contributions.
Should you have any questions, please feel free to contact the Track Chair at {ravi.chaturvedi1@sharda.ac.in},
Phone Numbe. :9873050339
Kindly send the acknowledgement to track chair email after uploading the revised paper at
CMT.
Note: All papers must be presented in the offline mode only.
Thank you once again, and Congratulations!

Reply | Reply all | Forward

PLAGIARISM REPORT

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.